Firmware, Arduino e il primo LED

Lezione 1 — 1IOT, SST-INF-01c — Dispositivi embedded, input e output, la scheda Arduino, i pin e GND, LED e resistore, Tinkercad, il primo programma

03/11/2026

1 Cosa vuol dire fare firmware

Fino a oggi avete scritto **programmi** che girano sul **computer**. Quello che imparate lì lo usate in ogni esercizio, e piano piano gli esercizi assomigliano sempre di più a programmi veri: è la parte di sviluppo delle applicazioni, quella del lunedì.

Fare **firmware** è una cosa leggermente diversa: il programma **non** viene eseguito dal computer, ma da un **dispositivo embedded**.

Definizione

Un **dispositivo embedded** è una macchina che fa calcoli come un computer, ma che **non** fa tutto quello che fa un computer: è costruita per eseguire **un solo compito**, o un gruppo di compiti molto ridotto. Di solito si presenta come una **scheda elettronica**.

Sul computer potete giocare, navigare, usare Excel, guardare video, usare i social. Il computer dentro il **frigorifero**, nel **forno** o nella **centralina dell'auto** è sempre un computer, ma su quello non potete pensare di giocare a un videogioco.

1.1 La scheda e la CPU

Un dispositivo embedded è una **scheda elettronica** su cui è montata una **CPU**. La **CPU** è il **cervello**: è il componente che fa i calcoli. Non va confusa con la **scheda madre**, che è la scheda nel suo insieme. Anche nel computer ci sono scheda madre e CPU; la differenza è lo scopo: il computer è costruito per farci un po' di tutto, la scheda embedded per eseguire una sola funzione.

Nota

Perché le schede sono spesso verdi? Sopra la scheda c'è uno strato di **vernice** che protegge le piste elettriche dall'**ossidazione**. La vernice viene tolta solo dove bisogna saldare i componenti, con un procedimento che usa la **luce** (vernici fotosensibili). Il verde è il **colore storico**. Oggi fare una scheda di un altro colore costa poco, quindi ne trovate anche blu, rosse, bianche, nere, gialle, viola; non però di qualunque tonalità, perché i colori disponibili sono pochi.

2 Un esempio: il forno di casa

Il forno è una **scatola** con una porta e una maniglia. Dentro, sul fondo e in alto, c'è una **resistenza**, chiamata anche **serpentina** per la sua forma: è un componente molto più grosso dei piccoli resistori che useremo noi. Quando le date corrente si **scalda**, e scaldandosi aumenta la temperatura del forno e cuoce il cibo.

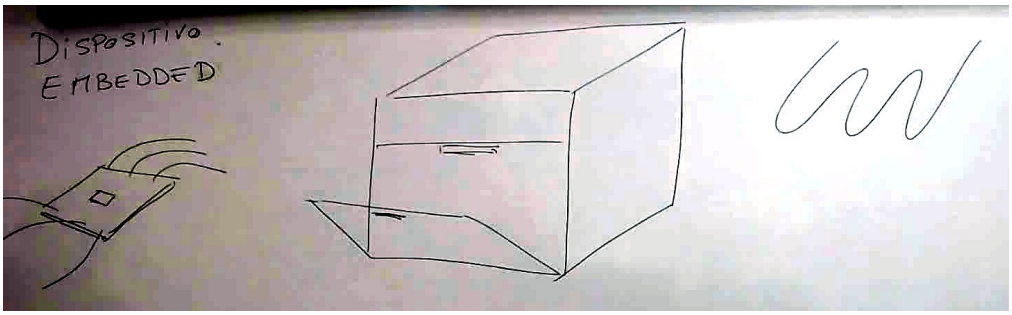


Figura 1: Alla lavagna: la scheda embedded con i suoi fili e il forno con la porta, la maniglia e la serpentina.

Il problema: se lasciate la resistenza accesa per dieci ore, la temperatura sale altissima e il cibo **brucia**. Serve un **controllo**:

- si accende la resistenza, che scalda;
- una **sonda** dentro il forno misura la temperatura;
- **se** la temperatura è arrivata a 180 o 200 gradi (dipende da cosa state cucinando), si **spegne** la resistenza;
- **se** la temperatura scende sotto quel valore, si **riaccende**.

Avete notato la parola **se**? Nel programma della scheda c'è proprio un `if`. E al posto di `printf` ci saranno dei comandi che accendono e spengono la resistenza. Questo è un **controllo automatico** di temperatura.

Lo stesso si può fare con il **frigorifero**, al contrario, o con la centralina di un **cancello** automatico: quando premi il pulsante del telecomando il cancello si apre, **se** non c'è nessuno in mezzo. Si parla quasi come in un programma: la scheda viene programmata per gestire proprio questo tipo di eventi.

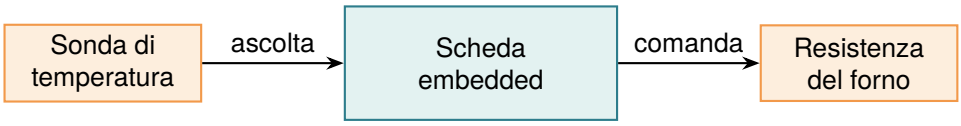


Figura 2: Il forno: la scheda ascolta la sonda e comanda la resistenza.

3 Input e output: i fili

Sul computer l'interfaccia è fatta di **tastiera** e **monitor**. Nelle schede embedded no: l'interfaccia sono **fili elettrici**, sui quali si può **mettere o togliere corrente**. Lo decide il programma: se un filo è collegato alla resistenza del forno, il programma può accenderla o spegnerla. Se invece a un altro filo è collegata la sonda, il programma può **ascoltare** cosa dice.

Ci sono quindi **due tipi di fili**: quelli che **entrano** nella scheda e quelli che **escono**. Sono sempre fili; cambia la **direzione dell'informazione**.

Esempio

Pensate di essere voi la scheda. La **bocca** serve a far **uscire** l'informazione: siete voi a decidere cosa dire. Le **orecchie** servono a farla **entrare**: non si producono suoni con le orecchie. Nelle schede funziona allo stesso modo: la scheda parla con il mondo esterno con alcuni fili e sente cosa dice il mondo esterno con altri.

Definizione

I fili che **entrano** nella scheda si chiamano **input**: su questi decide il mondo esterno e la scheda ascolta.
I fili che **escono** dalla scheda si chiamano **output**: su questi decide la scheda.

Questo schema lo troverete in qualunque documento: una scatola, la nostra scheda, con i fili di ingresso a **sinistra** e quelli di uscita a **destra**, perché di solito si legge da sinistra verso destra.

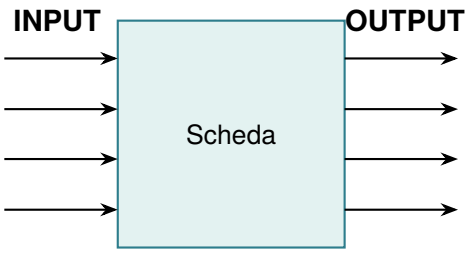


Figura 3: Lo schema disegnato alla lavagna: input a sinistra, output a destra.

Nota

Tenete un **quaderno per ogni materia**: uno per elettronica e uno per firmware, non appunti sparsi sui quaderni di altre materie. Se proprio volete usarne uno solo, giratelo al contrario e cominciate dal fondo.

4 La scheda Arduino

La scheda che useremo almeno per il primo anno è **Arduino**. Al centro c'è una scatolina nera con tante **zampette metalliche**: è il **chip**, e ogni zampetta è un filo che permette di comunicare con il mondo esterno. In realtà tutto Arduino sta in quel chip; viene montato su una scheda per **comodità**, perché da solo è piccolo e difficile da maneggiare.

Le zampette sono collegate ai **connettori** in alto e in basso sulla scheda: piccoli **forellini** in cui si infila un filo, che resta agganciato. Accanto ad Arduino useremo una piastra bianca, la **breadboard**, per montare i componenti, e dei fili per collegare resistori, lampadine, pulsanti: così si **espande** Arduino con le funzioni che servono.



Figura 4: Arduino Uno. In arancione i pin digitali, in verde i pin GND. Il chip nero è in basso a destra, il connettore USB in alto a sinistra.

4.1 Da dove arriva

Arduino nasce nel **2005** in **Italia**, a **Ivrea**, da un gruppo guidato da **Massimo Banzi**. Non è un'invenzione dal nulla: schede di questo tipo esistevano già. Il problema era la **difficoltà iniziale**, altissima:

- il chip si vendeva con un **manuale** di circa 200 pagine da studiare prima di poterlo programmare;
- non c'era YouTube e Internet era quello che era: il manuale spesso si comprava in libreria, anche per 80 euro, per programmare un chip che ne costava 2;
- servivano circa **200 righe di codice** solo per le impostazioni, prima di poter chiedere al chip di accendere una lampadina.

L'idea è stata preparare una scheda con un **programma di base** già dentro: la parte noiosa l'hanno fatta loro, e chi programma scrive poche righe per ottenere subito quello che gli serve. Il prezzo ha fatto il resto: circa **25 euro**, quando un sistema di programmazione simile poteva costare 800-900 euro. Poi sono arrivate le copie cinesi, vendute a 2-3 euro.

Attenzione

Il **mondo del lavoro** non funziona così. In azienda bisogna studiare il chip che si vuole usare e farlo funzionare, perché un Arduino dentro un prodotto in vendita ne farebbe **aumentare molto il costo**. Noi lo usiamo per **imparare** a ragionare come le macchine e prendere confidenza con le schede. Nei prossimi anni, un passo alla volta, vedremo i sistemi senza questa base già pronta.

Di modelli di Arduino ce ne sono **tantissimi**: più grandi, con più piedini, con chip più potenti, con connessioni WiFi, Bluetooth, GPRS, LoRa e altre ancora. Noi usiamo quello base, che ha le poche cose che ci servono all'inizio. Il prossimo anno ne vedremo uno con il WiFi.

4.2 USB e batteria

Arduino ha un connettore **USB** di tipo B, quello grande e vecchio; dall'altra parte del cavo c'è il tipo A, che va nel computer. Il cavo fa due cose:

- **alimenta** la scheda, cioè la accende;
- apre il canale per **scaricare il programma** nella memoria della scheda.

Il collegamento al computer **non è necessario** per il funzionamento: il programma **gira dentro la scheda**. Una volta programmata potete scollegarla, portarla in mezzo a un campo dove non c'è corrente, attaccare una **batteria** al suo connettore, e funziona. In laboratorio la lasceremo collegata alla USB per comodità.

5 I pin di Arduino

Sul connettore in alto ci sono i forellini numerati da **0** a **13**, con un buco libero, una divisione, tra il 7 e l'8. Attenzione: si parte da 0, quindi sono **14**. Questi forellini si possono chiamare in più modi: **pin**, **terminali**, **contatti**, **piedini**. Sono nomi diversi per la stessa cosa.

Prima abbiamo tenuto separati input e output. Arduino invece ha 14 pin che possiamo usare come vogliamo: **siamo noi a scegliere** quali sono input e quali output. Il 5 output, il 6 input, il 7 input, il 9 output: si decide nel programma.

Definizione

Arduino ha **14 pin digitali** (da 0 a 13), ciascuno utilizzabile come **input** oppure come **output**.

5.1 Cosa vuol dire digitale

Digitale vuol dire che il pin ha solo **due stati**: acceso o spento, cioè dà corrente o non la dà, come un interruttore della luce, come 1 o 0.

Esempio

Il **volume** del telefono o delle cuffie si regola: non è solo al massimo o spento. Questi pin non possono farlo: o sono accesi o sono spenti. La via di mezzo, "un po' acceso ma non troppo", **non esiste**.

- Sono io che decido se la lampadina è accesa o spenta: il pin è un **output**.
- Un sensore mi dice se la porta è aperta o chiusa, e io ascolto: il pin è un **input**.

5.2 GND: dove rientra la corrente

Prendiamo una **batteria** da 9 volt, con il suo più e il suo meno. Collegate il **più** a una lampadina, e la lampadina al **meno**: si accende. Avete creato un **circuito**: la corrente esce dal più, passa per la lampadina e torna al meno.

Con Arduino è uguale. Se la corrente esce da un pin (per esempio il 7), passa per la lampadina e deve rientrare da qualche parte. Quel punto è **GND**, che corrisponde al **meno della batteria**. GND sta per **ground**, cioè "terra". Sulla scheda ci sono tre pin GND: uno vicino al 13 e due sul connettore in basso.

Attenzione

Un dispositivo **non** si collega mai con un **solo filo**. Ne servono almeno **due**: uno porta la corrente e l'altro la riporta indietro, su **GND**. Senza il filo su GND la lampadina non funziona.

Perché si chiama terra

Il potenziale elettrico si misura rispetto a quello della **Terra**, che è il nostro **zero di riferimento**. Se aprite il quadro elettrico del laboratorio trovate cavi blu, neri e **gialloverdi**. Il gialloverde è il **cavo di terra**, legato alla sicurezza: finisce sottoterra, circa due metri sotto, su una piastra di metallo, e la corrente si disperde nel terreno, come un secchio d'acqua rovesciato che viene assorbito. Quel cavo è considerato il riferimento a **zero volt**; per questo il meno di una batteria e di una scheda viene chiamato **ground**, anche quando non è collegato davvero alla terra. Arduino lo scrive **GND**. Ha anche un simbolo, con alcune varianti: quello a sinistra è il più conosciuto e lo trovate anche sulle placche dei pali della luce.



Figura 5: I simboli di terra (ground).

Sicurezza sul lavoro

Voi state già a **zero volt**, perché siete appoggiati a terra. Ecco perché la **presa di casa** è pericolosa: se ci mettete un dito, la corrente passa **attraverso il corpo** per scaricarsi a terra, e si può **morire**. **Mai** toccare prese, quadri elettrici o cavi.

Esempio

Gli uccellini sui fili dell'alta tensione. Gli elettrodotti hanno tensioni altissime. Allora perché gli uccellini non muoiono? Perché toccano **un solo cavo** e non toccano terra: la corrente non ha un percorso da attraversare. Se una persona toccasse due cavi insieme, o un cavo e la terra, resterebbe uccisa all'istante. Per questo la manutenzione è complicatissima: bisogna assicurarsi di non toccare mai due punti insieme.

Sicurezza sul lavoro

Non ci si avvicina **mai** a una linea ad alta tensione: la scarica può partire anche **senza toccare** il cavo.

Nota

Il più e il meno che si scambiano. Nella rete elettrica la corrente è **alternata**: a 50 Hz un ciclo completo dura 20 millesimi di secondo, e il più e il meno **si scambiano** ogni 10 millesimi. È legato alla tecnologia con cui si trasporta l'energia.

Vicino agli elettrodotti. Sotto una linea ad alta tensione il **campo elettrico** è così intenso da far accendere un **tubo al neon** tenuto in mano, senza nessun filo collegato. Per legge intorno alle linee c'è una **fascia di rispetto** in cui non si possono costruire abitazioni: la sua larghezza dipende dalla linea.

6 LED e resistore

Cominceremo accendendo **lampadine**, perché è la cosa più semplice; poi aggiungeremo pulsanti e dispositivi via via più complessi. Il nome tecnico della nostra lampadina è **LED**: un componente che fa luce quando è attraversato dalla corrente e che **consuma pochissimo**. Quelli che useremo hanno un bulbo colorato (rosso, verde o giallo) e **due fili**, uno **lungo** e uno **corto**; la differenza di lunghezza è piccola, va guardata bene.

Attenzione

Un LED **non** si collega direttamente ad Arduino: la corrente sarebbe così grande da **bruciarlo**. Si passa sempre da un **resistore**.

Il **resistore** ha due terminali e **non ha un verso**: si può montare in un senso o nell'altro. Il **LED** invece **ha un verso**: se lo girate al contrario la corrente non passa e il circuito non funziona. Se qualcosa non va, controllate prima di tutto questo.

Definizione

Il filo **più lungo** del LED si chiama **anodo**: da lì la corrente **entra**.
Il filo **più corto** si chiama **catodo**: da lì la corrente **esce**.

I nomi servono davvero: nel simulatore, portando il mouse sopra un filo del LED, compare la scritta anodo o catodo.

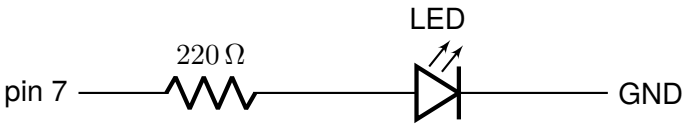


Figura 6: Il circuito: pin, resistore, LED (anodo verso il resistore, catodo verso GND).

Le **fasce colorate** indicano il valore del resistore, che si misura in **ohm** (Ω). Come si leggono lo vedrete nelle ore di elettronica. Più ohm ci sono, più il resistore "blocca" la corrente. Nei nostri esperimenti useremo il resistore da **220 Ω**: **rosso, rosso, marrone**, più una fascetta **oro** in fondo. Segnatevelo: quando prenderete in mano i resistori veri vi servirà.

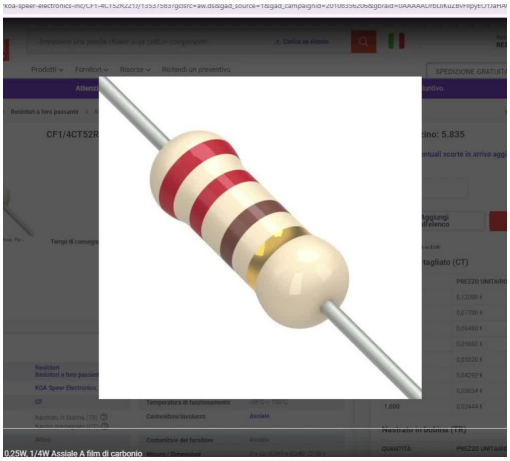


Figura 7: Un resistore con le fasce colorate.

All'inizio useremo un **simulatore**. Appena sarete più abili, l'idea è darvi una **scatola** con un Arduino, una breadboard, un mazzetto di fili, 10 LED e alcuni resistori, per realizzare i circuiti dal vivo. Il rischio è uno: lasciarla a casa la mattina di firmware. Quel giorno va portata a scuola.

7 Il simulatore: Tinkercad

7.1 Entrare in Tinkercad

1. Controllate che il browser sia collegato al vostro **account della scuola**. In alto a destra, vicino alla X di chiusura, può comparire il pulsante **Verifica la tua identità**: cliccateci e fate il login.
2. Cercate **Tinkercad**, con la K, ed entrate nel sito.

3. Scegliete **Iscriviti** (in alto a destra), poi **Crea account personale** (l'ultima voce) e **Accedi con Google**.
4. Scegliete **Italia** come paese, altrimenti non seguirete il percorso giusto, e inserite la data di nascita: per l'account serve avere almeno 14 anni.
5. Proseguite con **Avanti** e **Continua**.

7.2 Disegnare il circuito

1. In alto a destra scegliete **Crea**, poi **Circuiti**. A destra c'è l'elenco dei **componenti**, che si trascinano nel foglio.
2. Trascinate una **resistenza**. Cliccandola diventa blu intorno e in alto compare un piccolo menu per cambiare il valore. Di partenza è in **kΩ** (k vuol dire chilo, cioè mille): scrivete **220** e scegliete **Ω**. Il nome non importa. Se vedete una fascia gialla qualcosa è sbagliato: dev'essere rosso, rosso, marrone.
3. Trascinate un **LED**. Cliccandolo si sceglie il colore. Il colore del LED incide anche sul valore del resistore che serve: con il blu e il bianco, con questo resistore, potrebbe non funzionare. Per ora lasciate il **rosso**.
4. Portate il mouse sui due piedini del LED e leggete **anodo** e **catodo**.
5. Per non intrecciare i cavi, spostate il LED a sinistra e ruotate il resistore con il tasto **R**.
6. Per tirare un filo: **clic** sull'anodo, poi clic sul morsetto 1 del resistore, quello a sinistra. Il collegamento è fatto.
7. Trascinate **Arduino Uno R3**.
8. Collegate il **catodo** a **GND**. Selezionate il filo con un clic e dal menu in alto scegliete il colore **nero**.
9. Collegate l'altro morsetto del resistore al **pin 7**, con il colore che volete.

Nota

Il colore dei fili. Il filo di **GND** va sempre **nero**, quello di **alimentazione** sempre **rosso**. Se usate il verde il circuito funziona lo stesso, ma è una **convenzione**: chi deve guardare il vostro circuito, per esempio per la manutenzione, si aspetta di trovare così i fili. Per gli altri fili scegliete voi.

Attenzione

Nel circuito **non ci sono batterie**: non aggiungetene. La corrente la dà Arduino. Dal vivo una batteria collegata male **brucerebbe la scheda**: l'Arduino ve lo dà la scuola, ma se lo bruciate lo dovete ricomprare.

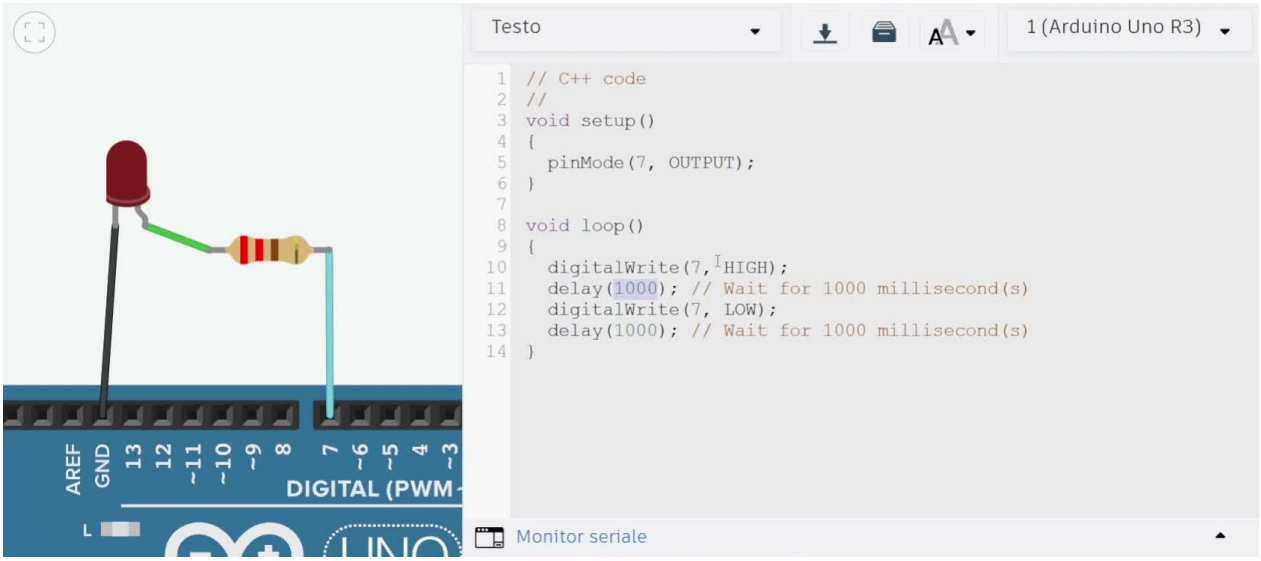


Figura 8: Il primo circuito in Tinkercad: LED e resistore sul pin 7, filo nero su GND, con il suo codice.

8 Il primo programma

Si preme il pulsante **Codice**. Tinkercad mostra i **blocchi**, pensati per i bambini delle elementari, ma noi scriviamo codice vero: nel menu in alto scegliamo **Testo** invece di Blocchi, e confermiamo con **Continua**. Compare un programma già pronto:

```
// C++ code
//
void setup()
{
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_BUILTIN, HIGH);
  // Wait for 1000 millisecond(s)
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  // Wait for 1000 millisecond(s)
  delay(1000);
}
```

`LED_BUILTIN` è il LED già montato sulla scheda. Noi invece abbiamo collegato il nostro LED al pin 7, quindi nei **tre punti** dove c'è scritto `LED_BUILTIN` scriviamo **7**. Se avete usato un altro pin, scrivete il suo numero. Le righe di commento (`// C++ code`, `// Wait for...`) non cambiano il funzionamento: qui sotto le abbiamo tolte per leggere meglio.

```
void setup()
{
  pinMode(7, OUTPUT);
}

void loop()
{
  digitalWrite(7, HIGH);
  delay(1000);
  digitalWrite(7, LOW);
  delay(1000);
}
```

Premete **Avvia simulazione**: il LED lampeggia. Per cambiare il codice bisogna prima premere **Interrompi simulazione**: finché il pulsante verde è acceso non si può modificare nulla.

8.1 Giocare con il tempo

Il numero dentro `delay` è in **millisecondi**: 1000 millisecondi sono un secondo, 3 secondi si scrivono **3000**, 10000 sono 10 secondi. Provate a cambiare i due valori:

- più il numero è **piccolo**, più il LED lampeggia **veloce**;
- scendendo verso 60-90 millisecondi si arriva al limite in cui l'occhio riesce ancora a vedere il lampeggio;
- con 10 millisecondi sembra **sempre acceso**, ma non è così: sta acceso 10 millisecondi e spento 10. È il nostro **occhio** che non riesce a vederlo.

8.2 Il diagramma del tempo

Questo è un **diagramma cartesiano**. Diagrammi come questo ne vedrete moltissimi, quindi imparate a leggerli. Sull'asse orizzontale scorre il **tempo**. Una linea **alta** vuol dire LED **acceso**, una linea **bassa** LED **spento**.

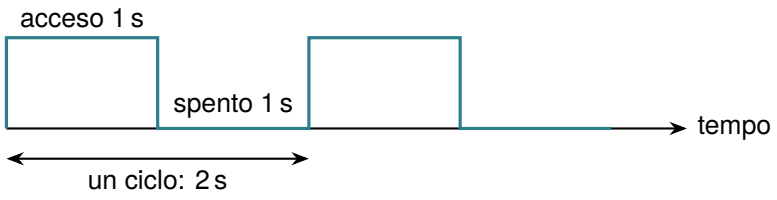


Figura 9: Il programma visto nel tempo: un secondo acceso, un secondo spento.

Leggiamo il codice, dentro `loop`:

- `digitalWrite(7, HIGH);` vuol dire "scrivi in digitale, sul pin 7, lo stato **alto**": dà corrente al pin, il LED si **accende**;
- `delay(1000);` la macchina resta **ferma** per 1000 millesimi di secondo, cioè un secondo: `delay` serve a **temporizzare** gli eventi;

- `digitalWrite(7, LOW);` scrive lo stato **basso**: il LED si **spegne**;
- `delay(1000);` altro secondo di attesa;
- arrivato all'ultima graffa, **ricomincia da capo**.

Un ciclo completo dura quindi **2 secondi**.

Nota

`digitalWrite` è un comando di Arduino, ed è un po' l'equivalente di `printf`: `printf` mostra il risultato sul monitor, `digitalWrite` lo mostra sul LED.

8.3 Cambiare un solo delay: la luminosità

Se abbassate **solo il primo** delay, il LED sta acceso poco e spento tanto. Questa è una tecnica che vedremo più avanti e che serve a **regolare la luminosità**. I LED non hanno una via di mezzo: sono sempre o accesi o spenti. Se scendete sotto i 20-30 millisecondi non vedete più il lampeggio: con 20 acceso e 20 spento il LED sembra fermo, solo un po' più debole. Le luci a LED regolabili funzionano così: **lampeggiano** così in fretta che l'occhio non se ne accorge, e cambiando quanto tempo stanno accese rispetto a quanto stanno spente si cambia la luminosità. Di solito, quando si accorcia il tempo in cui il LED sta acceso, si allunga quello in cui sta spento, così il ciclo dura sempre uguale.

8.4 I blocchi loop e setup

I comandi di `loop` stanno dentro due **graffe**: formano un **blocco** con un nome. L'argomento si chiama **funzioni** e lo faremo più avanti. Per ora pensate alle **macro** dei videogiochi: invece di premere destra, sinistra, sotto, sopra, pugno, calcio, si registra la sequenza su un solo pulsante. La funzione è più o meno così: un contenitore di comandi, come i **4 comandi** dentro il nostro `loop`.

Definizione

`loop` è il blocco che si **ripete continuamente**, finché la scheda è accesa. Lasciata accesa per dieci anni, lo ripete miliardi di volte.
`setup` è il blocco di sopra, fatto allo stesso modo, che viene eseguito **una sola volta** all'accensione. Per ora contiene un solo comando: `pinMode`.
`pinMode` dice se un pin è un **ingresso** o un'**uscita**. Per il momento li impostiamo sempre come uscite, con `OUTPUT`.

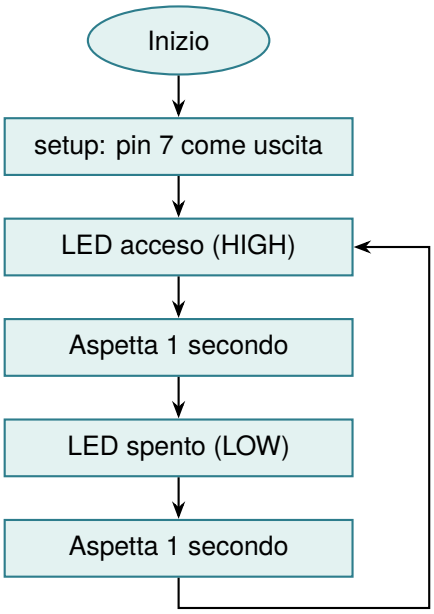


Figura 10: Il lampeggio: `loop` riparte sempre da capo.

9 Due LED insieme

Esercizio in aula: collegare un **secondo LED** su un altro pin, con il suo resistore da 220 Ω, e farlo lampeggiare insieme al primo. Ecco cosa serve, con il secondo LED sul **pin 3**:

1. in `setup`, `pinMode(3, OUTPUT);` perché Arduino deve sapere che anche il pin 3 è un'uscita;
2. in `loop`, dopo `digitalWrite(7, HIGH);` si aggiunge `digitalWrite(3, HIGH);`
3. sotto, dopo `digitalWrite(7, LOW);` si aggiunge `digitalWrite(3, LOW);`

```
void setup()
{
  pinMode(7, OUTPUT);
  pinMode(3, OUTPUT);
}

void loop()
{
  digitalWrite(7, HIGH);
  digitalWrite(3, HIGH);
  delay(1000);
  digitalWrite(7, LOW);
  digitalWrite(3, LOW);
  delay(1000);
}
```

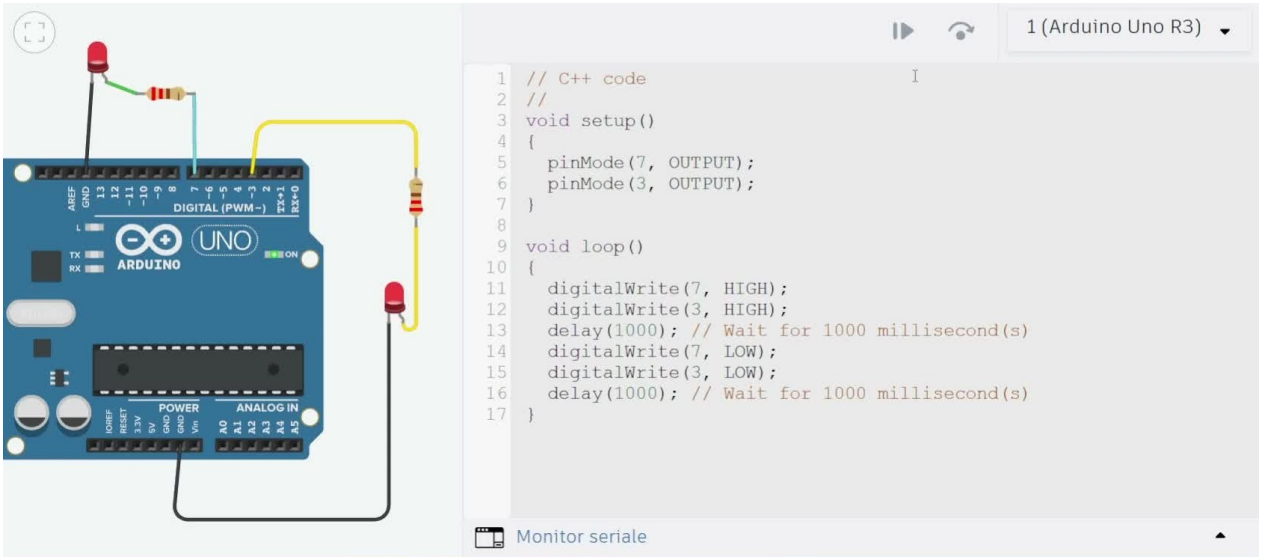


Figura 11: Due LED sui pin 7 e 3: i fili di ritorno sono neri e vanno su GND.

Attenzione

Errori da controllare:

- **pin diversi**: ogni LED ha il suo. Nel simulatore due LED sullo stesso pin sembrano funzionare, ma dal vivo non si deve fare, quindi ci si abitua subito nel modo giusto;
- **un pinMode per ogni pin**: se ne manca uno, quel LED non si comporta come dovrebbe;
- **setup e loop** sono uno solo per tipo: tutti i `pinMode` stanno nell'unico `setup`, tutti i `digitalWrite` e `delay` nell'unico `loop`;
- le **maiuscole**: `pinMode` con la M maiuscola, `OUTPUT` tutto maiuscolo;
- il filo di **ritorno** va su **GND**, di solito nero: uno dei GND va bene, il secondo LED torna sul GND del connettore in basso.

10 Per martedì prossimo

Esempio

Realizzate in Tinkercad un circuito con **due LED** che lampeggiano **uno sì e uno no**: non insieme, ma prima uno e poi l'altro. A casa non serve nient'altro che il **simulatore**, che **salva da solo** il lavoro. **Non c'è consegna**: lo farete vedere la prossima volta. Il testo è nella scheda **E01**.

11 Cosa ricordare di questa lezione

- Il **firmware** gira su un **dispositivo embedded**: una scheda con una CPU, fatta per un solo compito.

- **Input** = fili che entrano, la scheda ascolta. **Output** = fili che escono, la scheda decide.
- **Arduino** ha **14 pin digitali**, da 0 a 13: solo acceso o spento, input o output a scelta.
- **GND** è il meno: la corrente deve sempre poter rientrare. Servono almeno **due fili**.
- Il **LED** ha un verso (**anodo** lungo, **catodo** corto) e vuole sempre un **resistore**: 220 Ω, rosso, rosso, marrone.
- Fili: **nero** per GND, **rosso** per l'alimentazione.
- `setup` si esegue **una volta**, `loop` si ripete **all'infinito**.
- `pinMode(pin, OUTPUT)` imposta il pin, `digitalWrite(pin, HIGH)` o `LOW` accende o spegne, `delay(ms)` aspetta.